

ChargeOps Milestone 1

10-Minute English Video Demo Script

Presenter(s): [Team Member Name(s)]

Course / Section: [Course / Section]

Recording date: [Submission Date]

PURPOSE Demonstrate a complete local application—from browser interaction, through application logic, to persistent data—while explaining the business domain and the project’s cloud-ready technical dimensions.

Before recording

- Start the full local stack with `npm run dev:all` and open `http://localhost:5173`.
- Sign in with the Operations Manager demo account: ops / chargeops-demo.
- Keep the browser at a readable zoom level and close unrelated tabs or notifications.
- Have one sample technician photo or PDF ready if you plan to demonstrate Field uploads.
- Turn on the camera for every presenting team member during that person’s section.
- Rehearse once and target 9:30–10:00 so the final recording stays within the limit.

Run of show

Time	Screen / action	Message
0:00–0:50	Camera + title screen	Domain, problem, and project vision
0:50–1:30	Architecture orientation	Three local components working together
1:30–2:35	Login + Dashboard	Role-based access and operational overview
2:35–3:30	Stations + Chargers	Relational asset hierarchy and availability
3:30–4:55	Maintenance	Dispatch and resolve an incident
4:55–6:05	Sessions + Payments	Charging, billing, wallet, and invoices
6:05–7:25	Field uploads	Unstructured files processed asynchronously
7:25–8:35	Cloud ops	Queues, workers, retries, and traffic spikes
8:35–9:20	Daily simulation	Realistic local platform activity
9:20–10:00	Camera + closing	Milestone fit and cloud direction

0:00–0:50 — Opening: the domain and the problem

ON SCREEN *Camera on. Show the ChargeOps title or login screen. Keep your face visible while you introduce the project.*

Hello, my name is [Team Member Name], and this is ChargeOps, our EV charging network operations platform. ChargeOps is designed for the people who keep a charging network running: operations managers, field technicians, and read-only stakeholders.

The business problem is that charger status, maintenance incidents, charging sessions, payments, and field reports are closely connected, but they are often handled in separate tools. That separation makes it harder to see which charger needs attention, who is responsible, whether a charging session was billed correctly, and whether a station is ready for customers.

We chose this domain because it has a clear operational workflow and meaningful technical depth. It requires structured relational data, unstructured technician files, and asynchronous processing for billing, telemetry, and traffic spikes. Our goal is to give an internal operations team one reliable view of the network.

0:50–1:30 — Local architecture

ON SCREEN *Briefly show Slide 2, the README architecture section, or a simple architecture diagram. Then return to the application.*

For Milestone 1, every component runs locally. The front end is a React and Vite web application. The application layer is a Node.js and Express API with authentication, role-based authorization, queue adapters, and background workers. The primary data store is MySQL. The application also uses local object storage for photos, PDFs, and generated invoice files.

The browser never talks directly to the database. A user action calls the API, the API validates authorization and business rules, and then it reads or writes persistent data. Longer-running work is placed on a queue and completed by workers. That separation gives us a complete local system now and clear seams for later cloud migration.

1:30–2:35 — Login and dashboard

ON SCREEN *Sign in as ops using ops / chargeops-demo. Pause on the Dashboard and point to the KPI cards, charts, and recent activity.*

I will sign in as the Operations Manager. Authentication happens through the application layer, and the returned session determines which routes and actions this user can access. We also have a technician role for field work and a viewer role for read-only access.

The dashboard summarizes live operational data from MySQL. It shows the station and charger inventory, the user base, plan information, charger availability, revenue by city, and recent activity.

These values are not hard-coded cards; the front end requests them from the API, and the API aggregates the underlying relational tables.

This page gives an operations manager a fast answer to three questions: how large is the network, how much of it is available, and where should the team investigate next?

2:35–3:30 — Stations and chargers

ON SCREEN *Open Stations, search or filter once, and select a station. Then open Chargers and point out status, type, and station association.*

Stations and chargers demonstrate the asset hierarchy in our relational model. A company operates stations, and each station contains one or more chargers. The interface lets the operator search the network, inspect location and availability, and connect a physical charger to the sessions and incidents recorded against it.

The important point is consistency across the system. A charger displayed here is the same charger referenced by a charging session, a payment, and a maintenance record. Foreign-key relationships prevent the application from creating disconnected operational records.

3:30–4:55 — Core maintenance workflow

ON SCREEN *Open Maintenance. Choose an open item. Assign a technician from the dropdown, click Start, and then Resolve if you are comfortable changing the demo record. Show the resulting status after each action.*

Maintenance is one of our core user flows. An open incident identifies the charger, station, reported issue, assigned technician, status, and resolution time. As an operations manager, I can assign the correct technician. That change goes through an authorized API endpoint and is persisted in the maintenance log.

Next, I can start the work. The status changes from Open to In Progress. When the issue is complete, I can resolve it, and the system records the resolved time. These buttons are not only visual state changes: each transition is validated by the application layer and saved in MySQL.

Role-based controls matter here. An operations manager can coordinate the entire network. A technician sees the workflow needed for field service. A viewer can inspect information without changing it. This makes the product primarily an internal operations application, not a public driver-facing charging app.

4:55–6:05 — Sessions, payments, wallets, and invoices

ON SCREEN *Open Sessions and select a completed example. Then open Payments and show its matching payment or invoice metadata.*

A charging session connects a user, vehicle, charger, start and end time, energy consumption, and total cost. When a completed session is billed, the application creates a payment and updates the user wallet in one controlled workflow. Successful billing also produces invoice metadata and a PDF file.

This is where transactional integrity is important. We validate that a session is billed at most once, that payment status matches the session outcome, and that wallet balances do not become negative. We also repaired legacy demo data before this milestone by releasing stuck chargers, expiring stale subscriptions, cancelling completed-but-unpaid sessions, failing an abandoned pending payment, and restoring missing invoice metadata.

The final result is traceable from physical charging activity to a financial record. The operations team can investigate both service reliability and revenue without leaving the platform.

6:05–7:25 — Field uploads and unstructured files

ON SCREEN *Open Field uploads. Upload a prepared photo or PDF, or show an already processed upload if recording time is limited. Point out the asynchronous status and linked maintenance record.*

The relational database is not the only data source. Field technicians also upload photos and PDF reports. The API stores the file in local object storage, creates metadata, places a file-processing job on a queue, and immediately returns an accepted response instead of blocking the browser.

A background worker then parses the file. For images, it can inspect JPEG and EXIF metadata. For PDFs, it extracts text. The worker classifies the likely fault type, severity, and error code, compares GPS information with station locations when available, and then creates or updates the corresponding maintenance ticket.

This demonstrates our unstructured-file dimension and application logic beyond basic create, read, update, and delete screens. A technician's evidence becomes a searchable, actionable operational record.

7:25–8:35 — Asynchronous operations and traffic spikes

ON SCREEN *Open Cloud ops. Show queue depth, worker count, throughput or recent jobs. If the system is idle, explain that zero backlog is the healthy state.*

Cloud ops exposes the asynchronous part of the platform. Billing, file processing, telemetry, and simulation jobs are separated into queues. Workers claim jobs, update their progress, and complete them outside the request-response path.

The worker system includes retry delays, dead-letter handling for repeated failures, and idempotency protections so the same request does not create duplicate business records. A local autoscaling supervisor can increase or reduce worker capacity based on backlog. This is especially relevant to EV charging because device telemetry and session completion events can arrive in bursts.

For Milestone 1, the queue is implemented locally with MySQL-backed job records. The design is intentionally replaceable, so a later cloud version can map the same boundary to a managed queue and horizontally scaled workers.

8:35–9:20 — Daily operating simulation

ON SCREEN *Show the simulation status in Cloud ops, or briefly show the terminal command `npm run simulate:status`. Do not trigger a second run if today is already complete.*

To keep the demo environment representative of a normally operating platform, ChargeOps includes a daily simulator. A MySQL event schedules one run per date, and a worker generates realistic charging sessions, payments, invoices, telemetry, and maintenance situations.

The current run generated eighteen sessions and two maintenance situations. All eighteen payments and invoices completed successfully, with no negative wallets or duplicate billing. The simulator is bounded and idempotent, so it creates useful activity without continuously corrupting the data.

9:20–10:00 — Closing

ON SCREEN *Return to camera. Optionally keep the dashboard visible beside your camera feed.*

To summarize, ChargeOps is a working local application with all three required components. The React front end supports real operator workflows. The Express application layer enforces authentication, roles, business rules, and asynchronous jobs. MySQL and local object storage persist both structured and unstructured data.

We are proud of this domain because the user flow is easy to understand while the architecture addresses real cloud-computing concerns: relational consistency, file processing, and traffic spikes. We have also verified the project with a successful production build, twenty-six self-tests, and eighteen smoke tests.

In the next milestone, these local seams can move to managed cloud services for web delivery, containers, relational storage, object storage, and queues. Thank you for watching our ChargeOps Milestone 1 demonstration.

Presenter handoff and evidence checklist

- Replace every bracketed placeholder before recording or submission.
- Keep each presenting team member's face visible during that person's spoken portion.
- Show at least one write operation that visibly persists after refresh.
- Show the front end, explain the API/application logic, and identify MySQL plus object storage.

- Mention the three technical dimensions: relational data, unstructured files, and asynchronous traffic spikes.
- Include team collaboration evidence in the report or course submission area.